Detecting Anomalous Network Behavior in Defense Systems Using Time-Aware Feature Engineering

Ashley Figueroa, Cameron Aljilani, Juan Norena Murcia

Group 2

Shiley-Marcos School of Engineering, University of San Diego

AAI-501: Introduction to Artificial Intelligence

Andrew Van Benschoten, PhD

April 13, 2026

Abstract

This project developed supervised learning models to detect anomalous network behavior using the Kaggle Network Anomaly dataset. The dataset did not contain explicit timestamps, so we could not use a standard time-series approach. Instead, we ran a row-to-row difference analysis during exploratory data analysis and found that changes between adjacent observations were small and stable (mean difference near zero, standard deviation approximately 0.07). This allowed us to treat the row index as a temporal proxy. We then applied time-aware feature engineering by generating lagged variables, rolling statistics, and normalized deviation metrics (delta and z-score). These additions expanded the feature space from 5 to 29 columns while preventing data leakage.

We evaluated two classifiers: Logistic Regression and Gradient Boosting (n_estimators = 150, learning_rate = 0.05, max_depth = 3). The initial single stratified train/test split produced perfect performance for Gradient Boosting, which led us to also run Stratified K-Fold Cross Validation (k=5) with out-of-fold predictions. Gradient Boosting maintained exceptional performance (Accuracy 0.9958 ± 0.0041) compared with Logistic Regression (Accuracy 0.9709 ± 0.0101). The engineered features proved critical to improving both the single-split and K-Fold model performance, demonstrating that sequential structure can be extracted from non-temporal data and used for anomaly detection in network traffic.

1. Introduction

Network anomaly detection is one of the critical features used in modern cybersecurity and operational monitoring to detect adversarial attacks in critical networks including financial, national defense, and medical institutions. The ability to detect anomalous behavior in near-real time can prevent data exfiltration, distributed denial of service (DDoS) attacks, or attacks on network intrusion detection systems.

This project demonstrates binary classification of network traffic using a publicly available Kaggle dataset containing 1,654 observations. The data consists of four numerical features (inbound/outbound data rate and bandwidth utilization) and a binary label (0 = normal, 1 = anomalous) for network behavior (Amin, n.d.). Because the dataset does not contain explicit timestamps, the row index is used as a temporal proxy after the sequential ordering was confirmed through row-to-row difference analysis. The objectives of this project were to:

1. Develop time-aware features that capture short-term temporal dependencies
2. Compare Logistic Regression (baseline) with Gradient Boosting
3. Evaluate performance comparing both a single stratified split and Stratified K-Fold Cross-Validation to mitigate overfitting concerns

The solution demonstrates that engineered temporal features can yield high detection performance on static tabular data.

2. Data Description

The dataset consists of four numerical features: inbound rate, outbound rate, inbound bandwidth utilization, and outbound bandwidth utilization. The target variable is binary, where 0 represents normal network behavior and 1 represents anomalous network activity. All features are standardized, with a mean approximately equal to zero and a standard deviation near one,

indicating that the dataset has been preprocessed using normalization techniques. Standardization improves model convergence and comparability across features but removes direct interpretability of real-world metrics such as throughput in Mbps. The dataset is perfectly balanced (827 normal / 827 anomalous), which simplifies model training but does not reflect real-world cybersecurity environments where anomalous events are typically rare (i.e., a minority class).

3. Data Cleaning

The Kaggle Network Anomaly dataset was artificially clean and contained no missing values. For completeness, the initial data pre-processing included an explicit step to identify missing values in each data column. Any missing values would have been replaced with the median value for that feature (median imputation). The data types in the dataset were verified using the .info() command. In real-world data, human errors can lead to type errors. For example, a transposition error could result in a zero being typed as the letter "O." For this dataset, the inbound and outbound statistics were all "float64" while the class label was "int64." Had there been a transposition error as described, the data type (Dtype) would have been "object" indicating an issue. Later, during feature engineering, the creation of lag and rolling statistics produced NaN values in the first few rows of data because the algorithms could not identify past observations in the five step rolling window. These values were removed using the .dropna() command to maintain a clean dataset for modeling. For completeness in larger scale projects, additional cleaning steps would have included duplicate row detection, outlier analysis, or format validation (e.g., date, time formats).

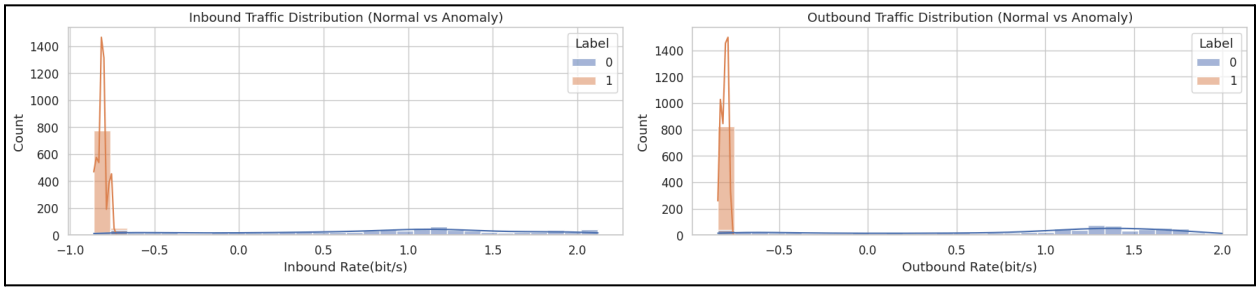
4. Exploratory Data Analysis

The purpose of exploratory data analysis (EDA) is to understand the structure,

relationships, and separability of features within the network anomaly dataset. This step is critical in determining whether machine learning models can effectively distinguish between normal network behavior and anomalous activity in a mission-critical cybersecurity context.

Feature Distributions

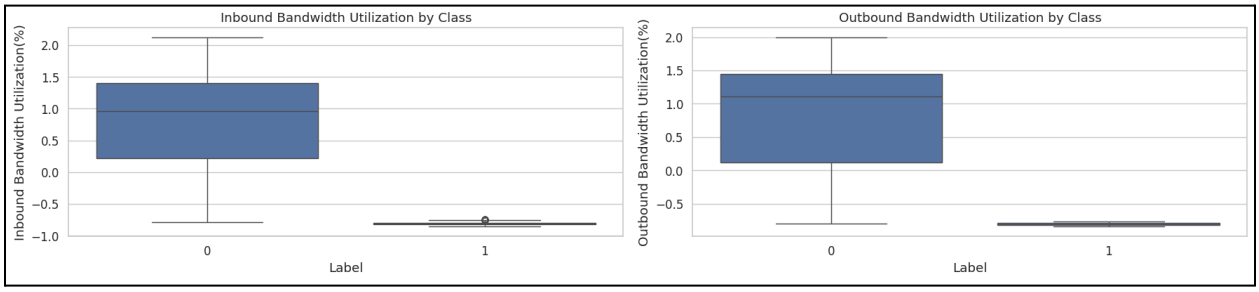
Analysis of the feature distributions indicates a clear differentiation between normal and anomalous traffic. Normal observations (Label = 0) are distributed evenly across higher values of inbound and outbound rates, while anomalous observations (Label = 1) are tightly grouped around lower negative values. This indicates that anomalies are associated with degraded traffic patterns rather than high-volume spikes.



Bandwidth Utilization Analysis

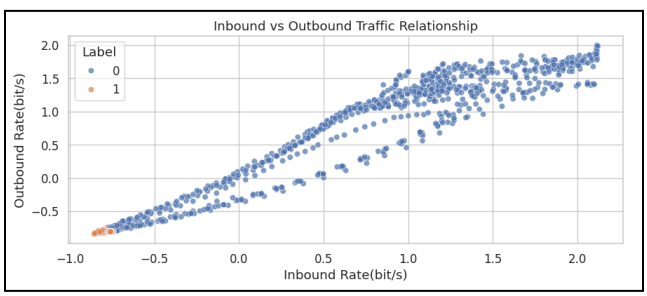
Our analysis using boxplots shows strong separation between data classes in bandwidth utilization. Normal traffic exhibits a broad spectrum of utilization values, while anomalous traffic is concentrated at lower utilization levels with minimal variability. The pattern suggests that anomalies may correspond to adverse network conditions indicated by link degradation,

partial loss of communication, or system instability.



Feature Relationships

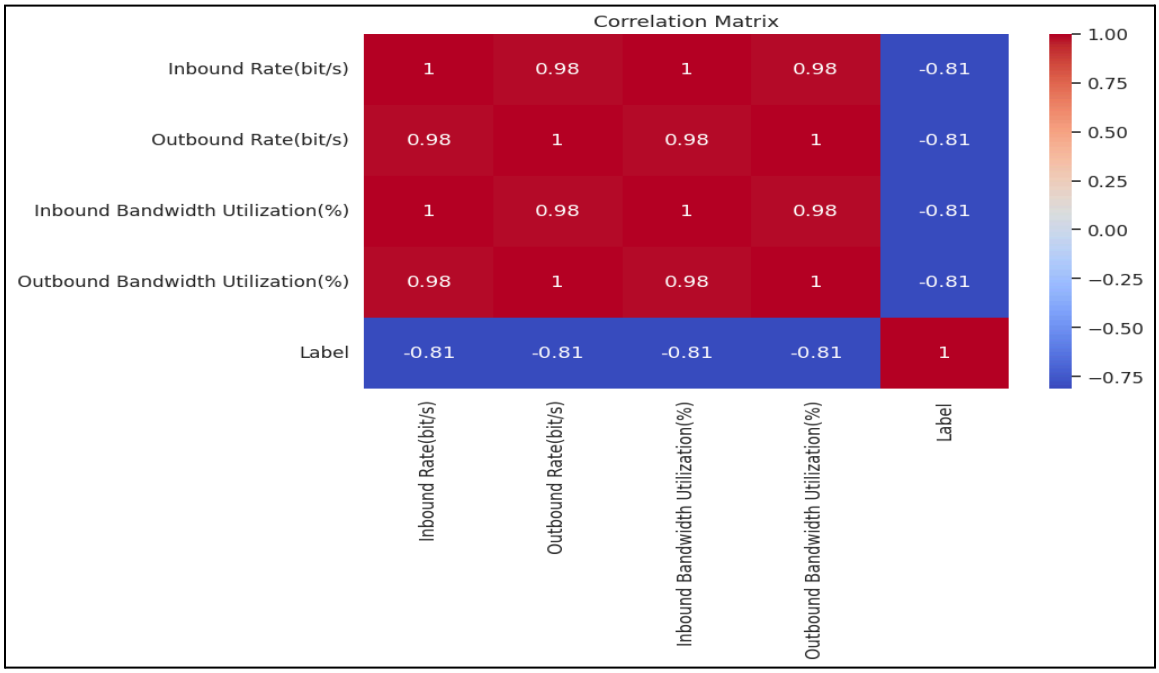
Scatter plot analysis of inbound versus outbound traffic demonstrates a strong linear relationship between these variables, particularly for normal observations. This behavior is consistent with expected network traffic and operations, where inbound and outbound flows are closely matched. Anomalous observations form an identifiable cluster in the lower range of the scatter plot, deviating from the linear trend created by the normal observations.



Correlation Analysis

The correlation matrix highlights two key findings. First, there is strong multicollinearity among the input features, with correlation values ranging from approximately 0.98 to 1.00. Second, all features exhibit a strong negative correlation with the target variable (approximately -0.81), indicating that anomalous observations are consistently associated with lower traffic and

utilization values.



Key Insights

The EDA reveals several important conclusions. The dataset is highly structured and exhibits clear separability between normal and anomalous classes. Anomalies are primarily characterized by reduced traffic and utilization rather than extreme spikes. Additionally, the presence of high multicollinearity indicates feature redundancy, which may influence model interpretability and selection.

Implications for Modeling

These findings directly inform the modeling approach. Linear models such as logistic regression may perform well due to strong separability, while tree-based models such as gradient boosting can effectively handle feature redundancy and nonlinear relationships. Given the mission-critical nature of the problem, emphasis was placed on minimizing false negatives, as missed anomalies pose significant security risks.

Operational Context

In real-world defense systems, including satellite communication (SATCOM) networks and unmanned aerial vehicle (UAV) data links, anomalies often manifest as sudden drops in throughput, abnormally low bandwidth utilization, or disruptions in inbound and outbound traffic symmetry. The patterns observed in this dataset align with these types of degradation and instability scenarios. For high-stakes Department of Defense missions, such anomalies can have severe operational and national-security consequences. For example, a sudden loss of SATCOM throughput in a contested electromagnetic environment (jamming) could sever command-and-control links during time-critical operations, resulting in loss of battlefield awareness or delayed response to threats. Additionally, anomalies in UAV data links could lead to loss of flight control, compromised targeting information, or mission failure, directly endangering personnel and mission objectives. These real-world risks underscore the critical need for accurate and timely anomaly detection to maintain operational superiority and protect service members.

5. Feature Engineering

Following the professor's guidance on avoiding formal time-series methods due to the lack of explicit timestamps, the team adopted a time-aware feature engineering approach using lags, rolling statistics, and deviation features. The professor confirmed this was a good approach provided the features were built carefully.

To address the absence of explicit temporal information, a row-to-row difference analysis was first conducted to evaluate whether adjacent observations exhibit continuity. The analysis showed small and stable differences between consecutive rows, with mean differences near zero and low variability. Based on this evidence, the row index was treated as a proxy for temporal

progression.

Lag Features

Lagged variables were introduced to capture prior system states by shifting each feature backward by one or more observations. These features let the model evaluate whether the current observation differs from recent history.

Rolling Statistical Features

Rolling statistics were computed to represent local behavior over a fixed window. Rolling mean and rolling standard deviation were used to capture the local baseline and short-term variability in network traffic. To prevent data leakage, we calculated rolling features using shifted values only.

Deviation Features

Deviation-based features were introduced to quantify how much each observation differs from its recent baseline. These include the difference between the current value and the rolling mean, as well as rolling z-scores that standardize deviations relative to recent variability. These features matter the most for anomaly detection, as anomalies often manifest as deviations from expected behavior rather than extreme standalone values.

Interpretation and Limitations

The engineered features transform the dataset from a static representation into a context-aware feature space. A key limitation of this approach is the absence of explicit timestamps. While the row index is used as a proxy for temporal order, the actual time intervals between observations are unknown.

6. Methodology

This study implemented a supervised binary classification pipeline using the

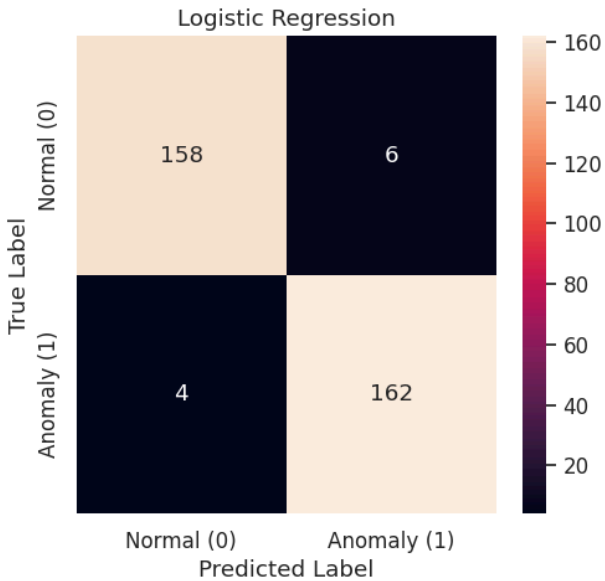
feature-engineered DataFrame networks_fe. After separating the engineered features from the target variable Label, two classifiers were evaluated. Logistic Regression was used as the baseline model with max_iter=1000 to ensure convergence. Gradient Boosting was selected as the main ensemble method with the hyperparameters n_estimators=150, learning_rate=0.05, max_depth=3, and random_state=42 (Pedregosa et al., 2011).

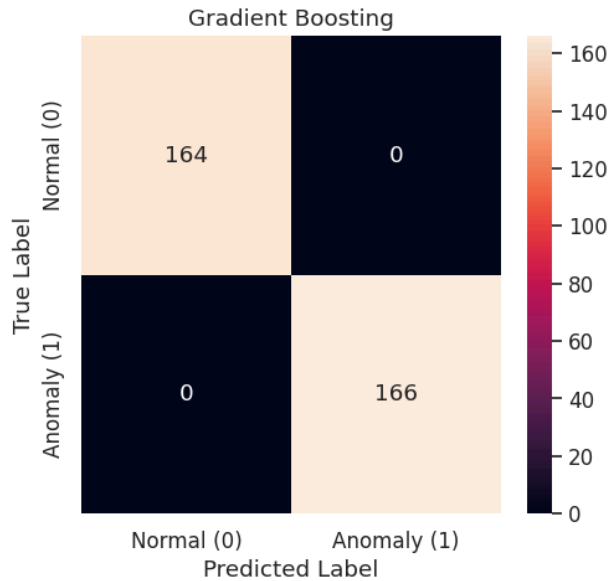
A stratified 80/20 train/test split was first applied to preserve the balanced class distribution. However, Gradient Boosting achieved perfect performance on this single split, which raised concerns from the team about possible overfitting or data leakage. To obtain a more reliable estimate, the evaluation was expanded to Stratified K-Fold Cross-Validation with k=5 and out-of-fold predictions. Stratified K-Fold Cross-Validation with k=5 divides the dataset into five equal folds while preserving the class distribution in each fold (Pedregosa et al., 2011). This approach ensured every observation was used for both training and validation while keeping the class balance consistent across folds.

The same preprocessing and evaluation framework was applied to both models for fair comparison. All modeling was performed on the time-aware feature set so the classifiers could take advantage of the short-term temporal dependencies created during feature engineering.

7. Model Evaluation

Two classifiers were evaluated using the time-aware feature set, Logistic Regression as the baseline and Gradient Boosting as the primary ensemble method. Performance was first assessed with a single stratified 80/20 train/test split. Gradient Boosting achieved perfect classification on this split (accuracy, precision, recall, and F1 score all equal to 1.0000), while Logistic Regression reached an accuracy of approximately 0.9697 with a small number of misclassifications.





To address concerns about potential overfitting and to obtain a more reliable estimate, the evaluation was expanded to Stratified K-Fold Cross-Validation with k=5 and out-of-fold (OOF) predictions. This approach ensured every observation contributed to both training and validation while preserving the balanced class distribution.

K-Fold Cross-Validation Results (k=5)

Logistic Regression

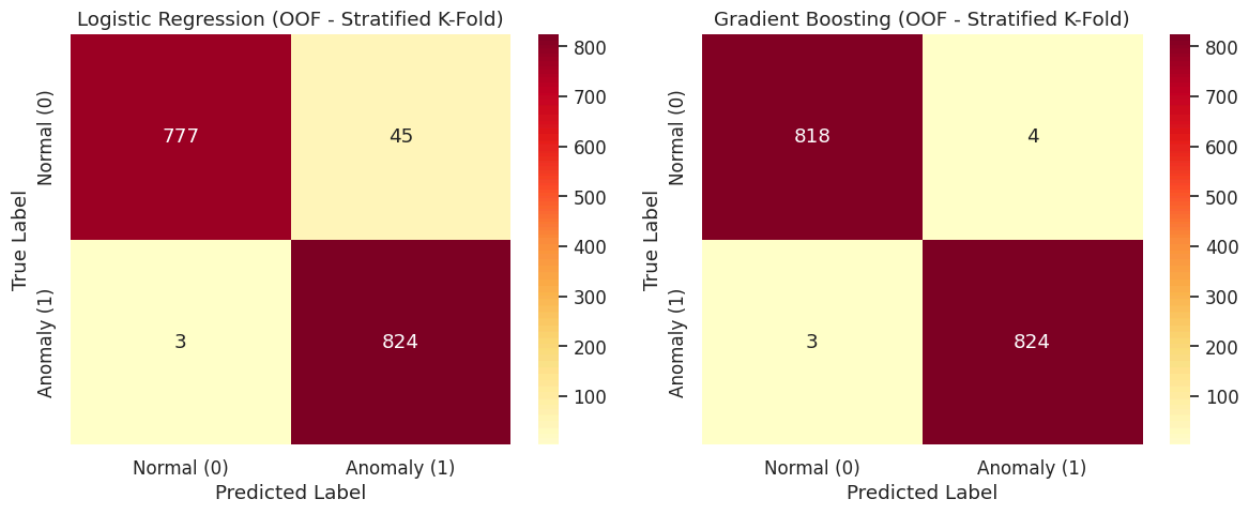
- Accuracy: 0.9709 ± 0.0101
- Precision: 0.9484 ± 0.0143
- Recall: 0.9964 ± 0.0048
- F1 Score: 0.9717 ± 0.0096
- ROC-AUC: 0.9939 ± 0.0038

Gradient Boosting

- Accuracy: 0.9958 ± 0.0041
- Precision: 0.9952 ± 0.0069
- Recall: 0.9964 ± 0.0048
- F1 Score: 0.9958 ± 0.0041
- ROC-AUC: 0.9994 ± 0.0013

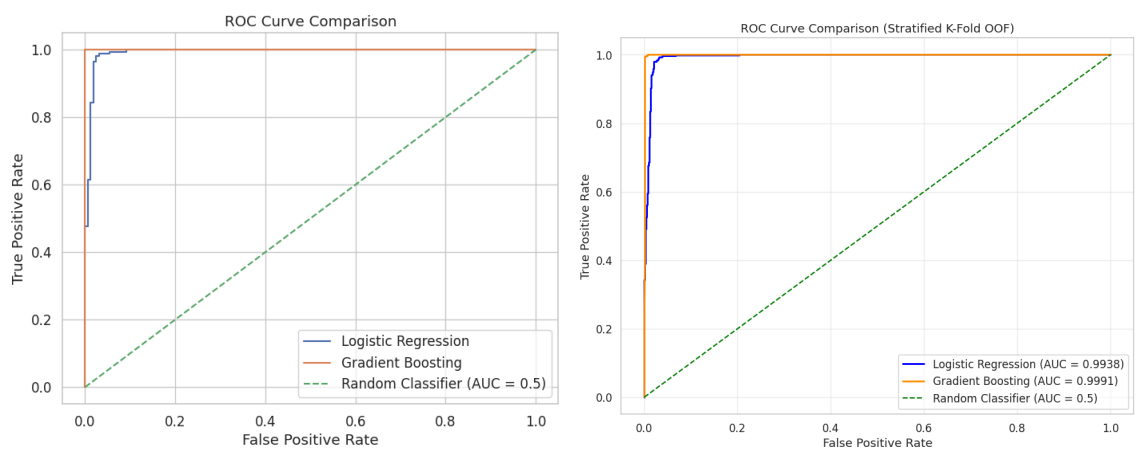
Out-of-Fold Confusion Matrices

The OOF confusion matrices confirmed very low error rates. Logistic Regression produced 45 false positives and 3 false negatives. Gradient Boosting performed exceptionally well, with only 4 false positives and 3 false negatives across the entire dataset.



ROC Curve Comparison

Both single-split and K-Fold OOF ROC curves showed strong discriminative power. Gradient Boosting maintained an AUC of approximately 0.9994 under K-Fold validation, while Logistic Regression achieved an AUC of approximately 0.9939. The curves for both models hugged the top-left corner of the plot, indicating excellent separation between normal and anomalous traffic. Overall, Gradient Boosting consistently outperformed Logistic Regression across all evaluation methods, demonstrating the value of the engineered time-aware features.



8. Discussion of Model Comparison

Gradient Boosting consistently outperformed Logistic Regression across all evaluation

criteria. The K-Fold results showed Gradient Boosting maintaining high performance with extremely low error rates, while Logistic Regression also performed well but with a slight but noticeable degradation in precision. The OOF confusion matrices and ROC curves confirmed that the time-aware features were highly effective in distinguishing normal from anomalous traffic.

The time-aware feature engineering approach was successful, enabling the models to capture short-term temporal dependencies in the static dataset. The low false-negative rate is particularly important for defense applications where missing an anomaly could have disastrous consequences. The primary limitation is the reliance on the row index as a temporal proxy, which may not fully capture true temporal dynamics in real-world data with varying intervals.

Overall, our project demonstrates that well-constructed feature engineering can extract meaningful sequential structure from non-temporal data, offering a practical approach to anomaly detection in mission-critical defense and aerospace systems.

9. Conclusion and Future Work

Our project successfully demonstrated that time-aware feature engineering can extract useful sequential information from a static dataset lacking explicit timestamps. By validating the row index as a temporal proxy and creating lag, rolling, and deviation features, we built a practical pipeline that allowed both Logistic Regression and Gradient Boosting to achieve excellent anomaly detection performance. Gradient Boosting stood out with very high cross-validated accuracy and a low false-negative rate, making it especially suitable for defense applications where missing an anomaly can result in high risks to military missions. These results confirm that careful feature engineering can turn a simple tabular dataset into one that captures short-term temporal dependencies.

To continue building on this initial framework, future work would include three main improvements: data augmentation; incorporating SHAP values; and testing the model on edge devices with limited space, weight, power, and compute (SWaP-C). Adding data augmentation during training could make the model more robust to variations in real network conditions. Incorporating SHAP values would help explain which features most influence the model's decisions, improving transparency and trust in human-machine teaming. Finally, testing the model on edge devices with quantization could show how well it performs under the size, weight, and power constraints typical of deployed defense systems.

Overall, this project showed that a thoughtful, non-formal time-series approach can deliver reliable anomaly detection. The team believes these methods offer a practical approach for similar problems in the cybersecurity and defense industry.

References

Amin, I. (n.d.). *Network anomaly dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/amineipad/network-anoamly-dataset/data

Chollet, F. (2015). *Keras*. https://keras.io

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv*. https://doi.org/10.48550/arXiv.1502.03167

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. http://jmlr.org/papers/v12/pedregosa11a.html